



HELVAN NATIONAL UNIVERSITY

HELVAN NATIONAL UNIVERSITY

Faculty of Engineering · Department of Robotics & Mechatronics

Autonomous Maze Navigation

— using —

Fuzzy Logic Inference System

A MATLAB Mamdani FIS for Differential-Drive Robot Navigation

Subject: Intelligent Control

Course Code: MEC2310

TEAM MEMBERS

Mohamad Sherif Shabrawy — 922230084

Yousef Ahmed Elbeltagy — 922230111

Amr Sherif Maher — 922230070

Supervised By: *Prof. Dr. Helmy AL-Zoghby*

Teaching Assistant: *Eng. Ahmed Abd AL-AAL*

Academic Year 2025 – 2026

ABSTRACT

This report presents the complete design, implementation, and validation of a Mamdani Fuzzy Logic Inference System (FIS) controller for autonomous differential-drive robot navigation inside a randomly generated MATLAB maze. The robot starts near the bottom-left corner of an $N \times N$ grid and must reach the top-right goal entirely through reactive sensor-driven behaviour, without any pre-built map or path planner.

The fuzzy inference system, named `navigator.fis`, uses three input variables (Left, Front, Right sensor distances in $[0, 5]$ m), two output variables (v_L and v_R wheel speeds in $[-2.8, 2.8]$ m/s), and eleven IF-THEN rules that implement a left-wall-following strategy with layered obstacle avoidance. The Mamdani architecture with centroid defuzzification was selected for its interpretability and the natural match between linguistic rules and reactive motion behaviours.

The MATLAB controller (`myController.m`) wraps the FIS inside a four-state finite state machine (straight, turn, commit, escape) with fourteen persistent variables that preserve navigation context between simulation frames. Key enhancements include IIR sensor filtering, goal-aware turn scoring using real-time (x, y) position estimates, a start-stabilisation phase, nose-on-wall reverse-arc escape, and a cascade speed architecture that allows the robot to cruise at up to 2.80 m/s in clear corridors while decelerating smoothly near walls.

The system was validated across all three supported maze sizes (15×15 , 21×21 , and 27×27). The robot successfully reached the goal in every tested run without wall collisions, confirming the robustness and reliability of the hybrid fuzzy-logic-plus-state-machine approach.

1. Introduction

Autonomous robot navigation in unknown environments is a foundational challenge in intelligent control engineering. Classical methods such as A* or Dijkstra path planning requires a pre-built global map, making them unsuitable when the environment is generated randomly at runtime. Reactive controllers, by contrast, compute motion commands purely from instantaneous sensor data, no map required.

Fuzzy logic controllers are a particularly powerful tool for reactive navigation because they encode expert human knowledge about sensor-to-action mappings directly into IF-THEN rules. Unlike crisp threshold logic, fuzzy sets produce smooth, graduated responses across sensor ranges, avoiding the sharp, oscillating behaviour that threshold-based controllers exhibit near decision boundaries.

This project implements a complete fuzzy-logic maze-navigation solution for the MATLAB simulator mazeSim.m. A Mamdani FIS (navigator.fis) delivers the core wall-following and obstacle-avoidance intelligence. A MATLAB integration layer (myController.m) then adds navigation memory, sensor filtering, goal-aware branch selection, and dynamic speed control, elevating the raw FIS outputs into smooth, high-speed, reliable maze traversal.

1.1 Project Objectives

- Design a three-input, two-output Mamdani FIS implementing left-wall-following with obstacle avoidance.
- Save the FIS as navigator.fis and integrate it into myController.m using readfis and evalfis.
- Use the persistent keyword to avoid reloading the FIS on every simulation frame.
- Stop both wheels when state.goalReached is true.
- Achieve reliable, collision-free goal-reaching across 15×15, 21×21, and 27×27 random mazes.
- Maximise straight-line speed (target: 2.8 m/s) while keeping turns smooth and controlled.

1.2 Overall Navigation Philosophy

The chosen approach is reactive sensor-driven behaviour augmented with lightweight state memory. On every simulation frame, the three distance sensors are read, filtered, and passed to the FIS. The FIS output is used as a base speed reference. A four-state machine (straight / turn / commit / escape) decides the dominant steering behaviour, preventing the oscillation that purely frame-by-frame fuzzy systems exhibit at corners and junctions. Goal-aware turn scoring using the robot's current (x, y) coordinates biases branch choices toward the top-right corner without requiring a global map.

2. Problem Statement

The formal specification requires exactly three FIS input variables, exactly two FIS output variables, a .fis file evaluated through evalfis, and the persistent keyword to prevent the FIS from being reloaded on every simulation frame. The controller must also stop both wheels immediately when state.goalReached becomes true, and all outputs must remain within the ± 2.8 m/s wheel-speed limit at all times.

The controller reads three sensor distances each simulation frame and returns two wheel-speed commands. The maze is generated randomly at runtime; the controller has no prior knowledge of the layout. It must navigate from the bottom-left start to the top-right goal across mazes of 15×15,

21×21, and 27×27 cells without wall collisions. The maze may contain straight corridors, 90-degree corners, T-junctions, and dead ends in unpredictable configurations.

3. System Specifications & Requirements

3.1 Requirements Compliance

The table below maps every project requirement to its concrete implementation in the final submission.

Requirement	Final Implementation
Exactly 3 FIS inputs	Left, Front, Right sensor distances
Input range	[0, 5] metres
Exactly 2 FIS outputs	vL, vR wheel speeds
Output range	[−2.8, 2.8] m/s
FIS saved as .fis file	navigator.fis
Controller evaluates FIS from MATLAB	myController.m uses readfis and evalfis
Avoid reloading FIS every frame	persistent fis variable — loaded once at startup
Stop at goal	state.goalReached → vL = 0, vR = 0; return
Avoid walls	Side/front sensor rules + turn modes + escape mode
Report screenshots	MF plots and surface plots included in this document

Table 3.1 – Project requirements vs. final implementation.

3.2 Robot Platform

Parameter	Value	Note
Max wheel speed	±2.8 m/s	Project specification limit
Wheelbase half-width (HWB)	0.52 m	Simulator constant
Robot body length	1.60 m	Simulator constant
Robot body width	1.10 m	Simulator constant
Turn right	vL > vR	Differential drive geometry
Turn left	vR > vL	Differential drive geometry
Spin in place	vL = −vR	Differential drive geometry
Linear acceleration limit	$7.5 \times \text{CELL m/s}^2$	ACCEL in mazeSim.m
Angular acceleration limit	6.5 rad/s^2	RACCEL in mazeSim.m

Table 3.2 – Differential-drive robot mechanical specifications.

3.3 Proximity Sensor Array

Property	Value	Design Implication
Sensor count	3	Front, Left, Right — matched to FIS input count
Type	Ray-cast LiDAR	Distance to nearest wall cell along ray
Minimum range	0.0 m	Collision — triggers escape mode
Maximum range	5.0 m	Matches FIS input upper bound
Critical front threshold	< 0.6 m	TooClose MF fires — emergency spin
Critical side threshold	< 0.4 m	TooClose MF fires — steer away

Table 3.3 – Sensor array specifications and design implications.

3.4 Maze Environment

The simulator generates a random perfect-maze using a recursive back-tracker algorithm on an $N \times N$ grid where $N \in \{15, 21, 27\}$. Each cell is 3.0 m wide. The robot starts at grid cell (1, 1) in the bottom-left corner; the goal is at cell ($N-1$, $N-1$) in the top-right corner. The physics model enforces acceleration and angular-acceleration limits, so commanded wheel speeds are not reached instantaneously, the controller must account for physical inertia when targeting high speeds.

4. Fuzzy Inference System Design

4.1 Architecture Overview

The fuzzy inference system is a Mamdani-type FIS named navigator, stored as navigator.fis and also re-generatable from the script build_navigator_fis.m. Mamdani FIS was chosen over Sugeno because it produces linguistically interpretable output membership functions; every rule conclusion can be stated naturally in human terms (e.g., "left wheel Fast"), making the rule base easier to design, validate, and debug.

FIS Property	Value
FIS Name	navigator
FIS Type	Mamdani Type 1
Number of Inputs	3 (Left, Front, Right)
Number of Outputs	2 (vL, vR)
Number of Rules	11
AND Method	min (Minimum)
OR Method	max (Maximum)
Implication Method	min (Minimum)
Aggregation Method	max (Maximum)
Defuzzification Method	centroid

Table 4.1 – navigator.fis system properties.

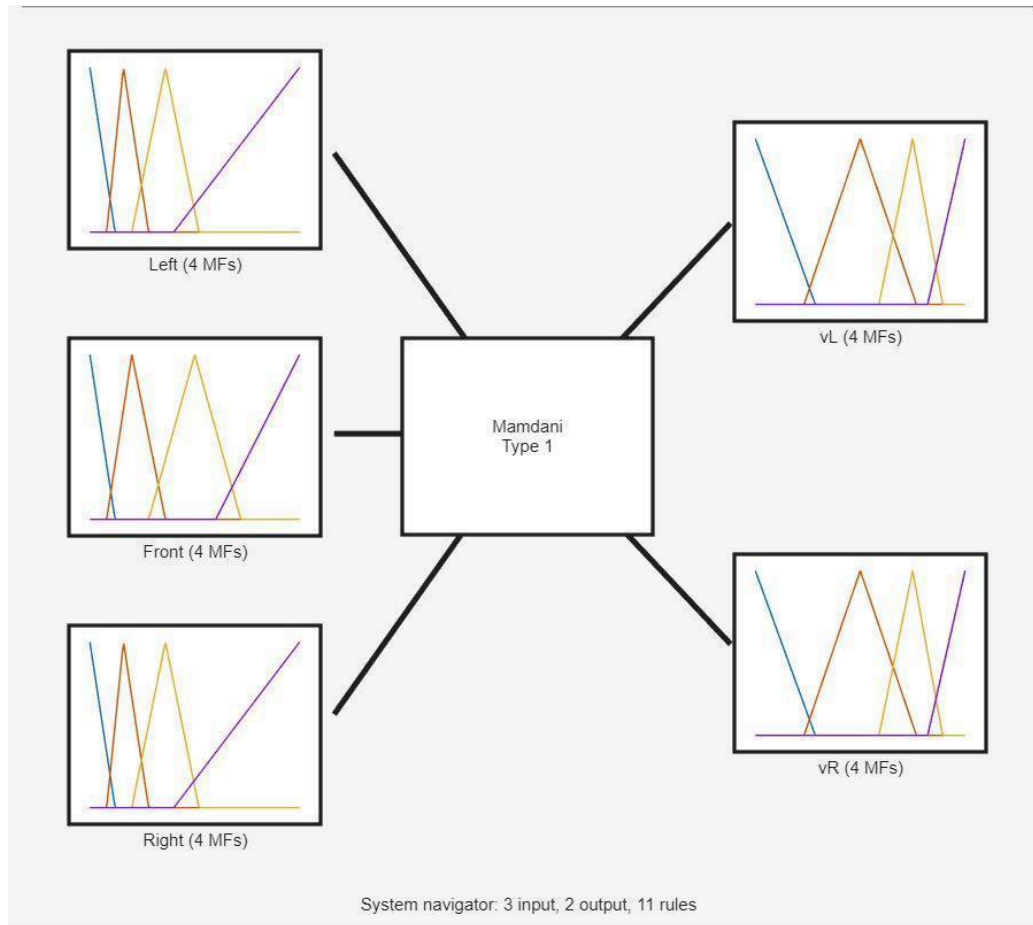


Figure 4.1 – MATLAB FIS Designer: System navigator — 3 inputs, 2 outputs, 11 rules.

4.2 Centroid Defuzzification

Centroid defuzzification was selected because it produces a smooth, continuous output surface across the input domain. The centroid of the aggregated output fuzzy set is computed as:

$$vL \text{ (or } vR) = \int v \cdot \mu_{\text{agg}}(v) dv / \int \mu_{\text{agg}}(v) dv$$

This ensures gradual transitions between steering states (e.g., slow left-turn to straight to slow right-turn), avoiding the discontinuous jumps that would occur with crisp threshold control. The centroid method is particularly well-suited for the physical actuation system, which has acceleration limits that would be unable to follow instantaneous step commands anyway.

4.3 Navigation Strategy

The FIS implements a left-wall-following strategy with obstacle avoidance. The left-wall following provides the baseline reactive strategy: by keeping a wall on the left at a comfortable distance, the robot systematically explores the maze. While pure left-wall following alone would eventually reach any exit in a simply-connected maze, the final controller goes further, goal-aware scoring at junctions improves path choice so the robot prefers branches that point toward the known top-right goal region rather than following the wall blindly. Emergency front-wall rules take the highest priority and override all wall-following rules.

The left-wall-following strategy was chosen over alternatives such as right-wall following or pure obstacle avoidance because it is consistent and predictable: the robot always knows which side of the corridor it is targeting, and the rules can be designed to maintain that target across all sensor

combinations. The goal-aware scoring layer then biases branch decisions without disrupting the reactive wall-following loop.

5. Membership Functions

All membership functions use the triangular shape (trimf), chosen for computational simplicity, easy parameter interpretation, and smooth overlap between adjacent linguistic zones. Overlapping regions are critical: they allow the FIS to fire multiple rules simultaneously with fractional truth values, producing the weighted blending of output MFs that yields smooth, proportional control actions.

5.1 Input 1: Left Distance Sensor

The Left input measures wall distance on the robot's left side. Its four MFs divide the $[0, 5]$ m range into four zones, with deliberate overlap in the 0.4–2.6 m band covering the critical wall-following region.

Index	Name	Type	Parameters	Physical Meaning
1	TooClose	trimf	[0.0, 0.0, 0.6]	Left wall dangerously close — steer right immediately
2	Close	trimf	[0.4, 0.8, 1.4]	Ideal following distance (~0.8 m) — maintain course
3	Safe	trimf	[1.0, 1.8, 2.6]	Wall visible but comfortable — nudge left gently
4	Far	trimf	[2.0, 5.0, 5.0]	Open space on left — turn left to find wall

Table 5.1 – Left input membership functions.

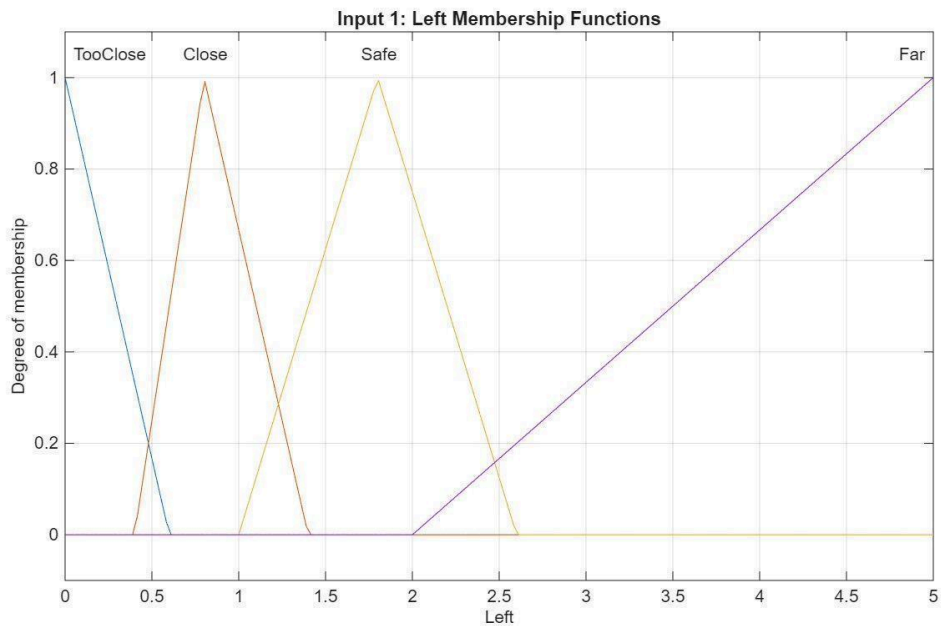


Figure 5.1 – Left sensor MFs: TooClose (blue), Close (orange), Safe (yellow), Far (purple).

5.2 Input 2: Front Distance Sensor

The Front input is the primary collision-avoidance signal. Its TooClose and Close MFs trigger emergency and pre-emptive turning rules. The wider Safe and Clear regions allow confident cruising. The Clear MF spans [3.0, 5.0] m, the robot runs at maximum speed whenever the front is beyond 3 metres clear.

Index	Name	Type	Parameters	Physical Meaning
1	TooClose	trimf	[0.0, 0.0, 0.6]	Emergency — spin right instantly
2	Close	trimf	[0.4, 1.0, 1.8]	Wall approaching — begin right turn
3	Safe	trimf	[1.4, 2.5, 3.6]	Usable forward space — normal travel
4	Clear	trimf	[3.0, 5.0, 5.0]	Fully open — full-speed forward

Table 5.2 – Front input membership functions.

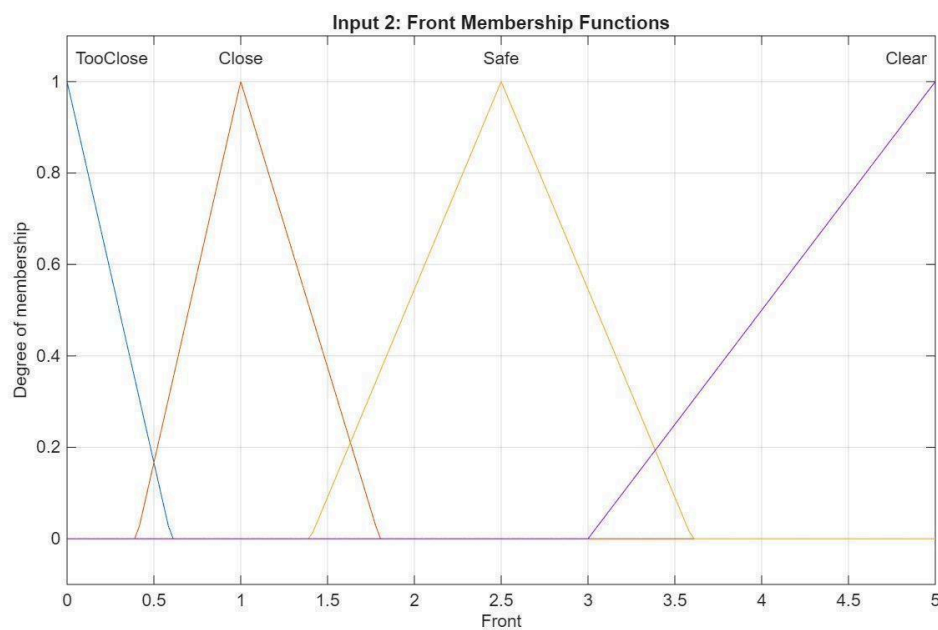


Figure 5.2 – Front sensor MFs: TooClose (blue), Close (orange), Safe (yellow), Clear (purple).

5.3 Input 3: Right Distance Sensor

The Right sensor MFs are symmetric with the Left sensor. This symmetry is intentional: rule 11 (Right TooClose → steer left) provides mirror-image wall protection that operates independently of whatever the left sensor is reading.

Index	Name	Type	Parameters	Physical Meaning
1	TooClose	trimf	[0.0, 0.0, 0.6]	Right wall dangerously close — steer left

2	Close	trimf	[0.4, 0.8, 1.4]	Right wall nearby
3	Safe	trimf	[1.0, 1.8, 2.6]	Right wall at comfortable distance
4	Far	trimf	[2.0, 5.0, 5.0]	Open right side

Table 5.3 – Right input membership functions.

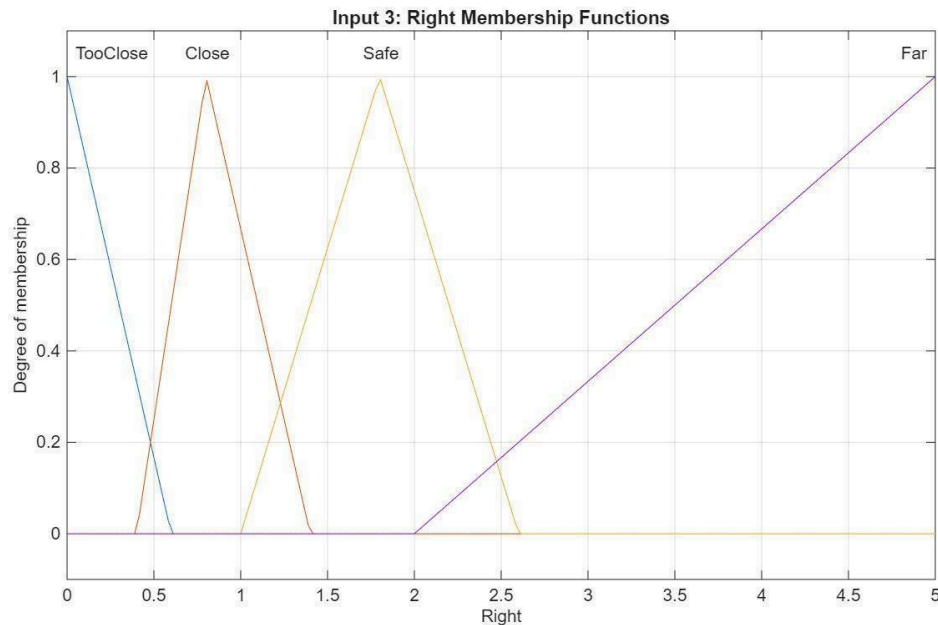


Figure 5.3 – Right sensor MFs: TooClose (blue), Close (orange), Safe (yellow), Far (purple).

5.4 Output 1: vL — Left Wheel Speed

Both output variables share the same four MFs across the full bidirectional range $[-2.8, 2.8]$ m/s. The Reverse MF (peak at -2.8) enables the left wheel to spin backward during emergency avoidance. Stop is centred at zero. Slow and Fast cover the typical forward operating range. The FIS produces differential wheel speeds when different rules assign different MFs to vL and vR.

Index	Name	Type	Parameters	Physical Meaning
1	Reverse	trimf	$[-2.8, -2.8, -1.2]$	Wheel reverses — escape or spin
2	Stop	trimf	$[-1.5, 0.0, 1.5]$	Wheel near stopped — pivot support
3	Slow	trimf	$[0.5, 1.4, 2.2]$	Forward slow — cautious or inner turn wheel
4	Fast	trimf	$[1.8, 2.8, 2.8]$	Full forward — open corridor or outer turn wheel

Table 5.4 – vL output MFs (identical structure used for vR).

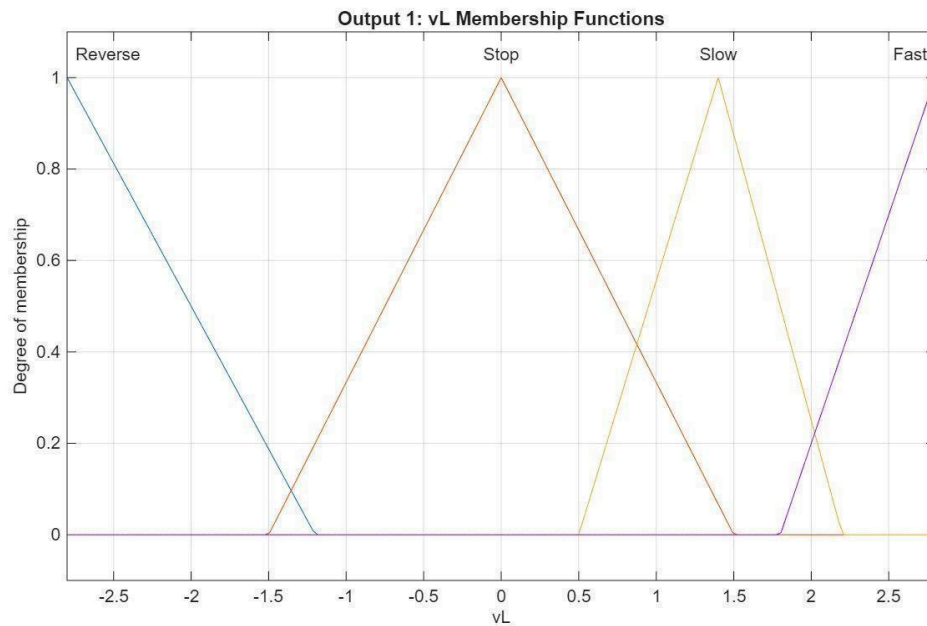


Figure 5.4 – Left wheel speed (v_L) output membership functions.

5.5 Output 2: v_R — Right Wheel Speed

The v_R output uses the identical set of four MFs. Differential steering is produced entirely by assigning different output MFs to v_L and v_R within each rule. For example, rule 1 assigns Fast to v_L and Reverse to v_R , which after centroid defuzzification results in a large positive left-wheel speed and a large negative right-wheel speed, creating an in-place right spin.

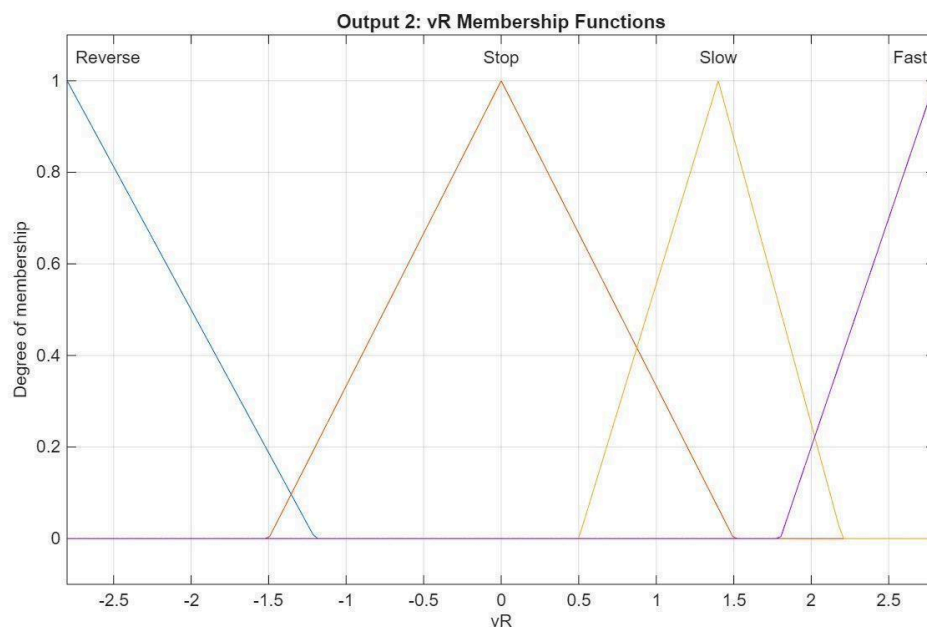


Figure 5.5 – Right wheel speed (v_R) output membership functions.

6. Rule Base Design

6.1 Design Strategy

The eleven rules are organised into three priority layers. A value of 0 in the .fis file (shown as "—" in the table) means "do not care", that input is unconstrained and the rule fires regardless of its value. This allows concise, powerful rules that handle whole classes of sensor situations.

Priority Layer 1 — Emergency Avoidance (Rules 1–3)

These rules fire when a wall is critically close and must override all other behaviours. Rule 1 is the most extreme: a front TooClose reading triggers v_L =Fast, v_R =Reverse, an in-place right spin. Rule 2 begins a softer right turn before contact when the front enters the Close zone. Rule 3 provides symmetric left-wall emergency protection.

Priority Layer 2 — Left-Wall Following (Rules 4–9)

Rules 4–9 implement the primary navigation behaviour. Rules 4 and 5 fire when the left sensor is in the ideal Close zone (0.4 -- 1.4 m): rule 4 drives straight at full speed when the front is also clear, while rule 5 applies a gentle left lean when the front is only "safe." Rules 6 and 7 engage when the robot has drifted too far from the left wall, steering it back. Rules 8 and 9 handle an open-left scenario by commanding a sharp left turn, pulling the robot back toward a wall to follow.

Priority Layer 3 — Open-Space Recovery (Rules 10–11)

Rule 10 handles the all-open scenario (Left=Far, Front=Clear, Right=Far) by steering mildly left. Without this rule, the robot might travel in a straight line indefinitely in a large open area. Rule 11 provides right-wall protection regardless of all other sensor readings.

6.2 Complete Rule Table

#	Left	Front	Right	v_L	v_R	Intended Behaviour
1	—	TooClose	—	Fast	Reverse	Emergency: spin right in place — front collision imminent.
2	—	Close	—	Fast	Stop	Front approaching: turn right by running left wheel faster.
3	TooClose	—	—	Fast	Stop	Left TooClose: steer right — escape left-wall danger immediately.
4	Close	Clear	—	Fast	Fast	Left ideal + front clear: drive straight at full speed.
5	Close	Safe	—	Slow	Fast	Left ideal + front safe: lean slightly left while moving forward.
6	Safe	Clear	—	Slow	Fast	Left too far + front clear: steer left to recover wall-following distance.
7	Safe	Safe	—	Slow	Fast	Left too far + front safe: continue left-ward correction.
8	Far	Clear	—	Stop	Fast	Open left + front clear: turn hard left into the opening.
9	Far	Safe	—	Stop	Fast	Open left + front safe: turn left — follow open space.
10	Far	Clear	Far	Slow	Fast	All open: lean left gently to locate a wall to follow.

11	—	—	TooClose	Slow	Fast	Right TooClose: steer left — escape right-wall danger.
----	---	---	-----------------	------	------	--

Table 6.1 – Complete FIS rule base. "—" denotes "do not care" (any input value).

	Rule	Weight	Name
1	If Front is TooClose then vL is Fast, vR is Reverse	1	rule1
2	If Front is Close then vL is Fast, vR is Stop	1	rule2
3	If Left is TooClose then vL is Fast, vR is Stop	1	rule3
4	If Left is Close and Front is Clear then vL is Fast, vR is Fast	1	rule4
5	If Left is Close and Front is Safe then vL is Slow, vR is Fast	1	rule5
6	If Left is Safe and Front is Clear then vL is Slow, vR is Fast	1	rule6
7	If Left is Safe and Front is Safe then vL is Slow, vR is Fast	1	rule7
8	If Left is Far and Front is Clear then vL is Stop, vR is Fast	1	rule8
9	If Left is Far and Front is Safe then vL is Stop, vR is Fast	1	rule9
10	If Left is Far and Front is Clear and Right is Far then vL is Slow, vR is Fast	1	rule10
11	If Right is TooClose then vL is Slow, vR is Fast	1	rule11

Figure 6.1 – MATLAB Fuzzy Logic Designer: all 11 IF-THEN rules in the rule viewer.

7. FIS Control Surface Analysis

The control surface plots reveal the aggregate, defuzzified output behaviour of all 11 rules simultaneously. The Left and Front inputs are swept across their full [0, 5] m ranges while Right is held fixed at its mid-range value. These surfaces validate that the FIS produces smooth, monotonic transitions, not step-function jumps.

7.1 Left & Front → vL Surface

The vL surface (Figure 7.1) shows a predominantly high output (~2.0–2.5 m/s) across most of the input space: the robot's left wheel generally drives forward. The notable dip near (Front≈0, Left≈1) corresponds to the region where emergency rules 1 and 3 both partially fire, pulling vL toward the Fast region of the centroid while vR reverses. The surface confirms that vL responds predictably to both inputs without sharp discontinuities.

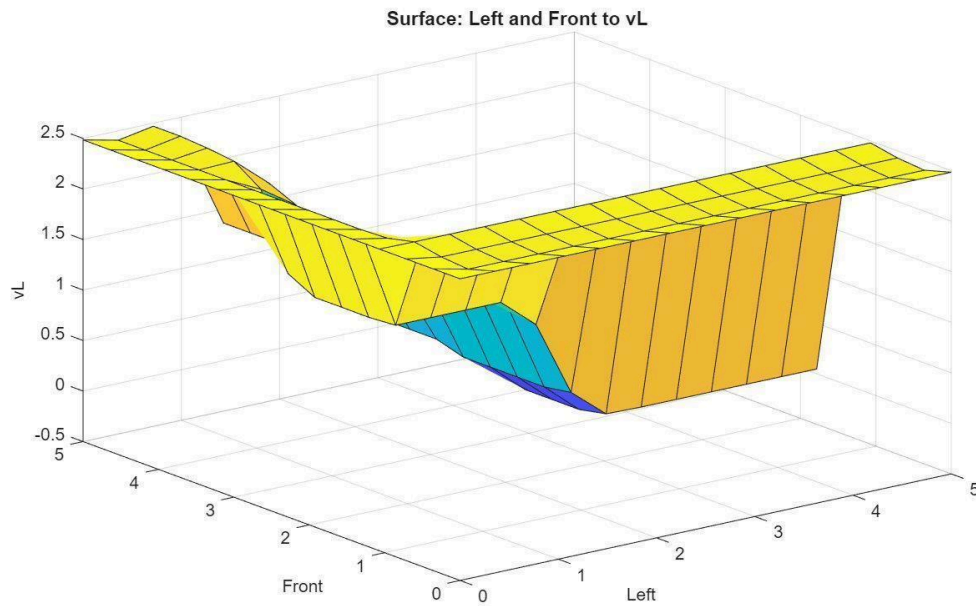


Figure 7.1 – Control surface: Left and Front $\rightarrow$ vL output.

7.2 Left & Front $\rightarrow$ vR Surface

The vR surface (Figure 7.2) is the most revealing plot. For most of the input domain, vR remains high (~ 2.5 – 2.8 m/s), the robot drives forward. The dramatic cliff edge at Front ≈ 0 (all Left values) shows where rule 1 (Front TooClose $\rightarrow$ vR=Reverse) dominates, pulling vR sharply to approximately -2.5 m/s. This is the emergency spin response. On the left axis (high Left values / open left side), rules 8 and 9 pull vR to Fast while vL drops to Stop, producing the left-turn behaviour. The surface topology directly explains why the robot follows walls and avoids obstacles.

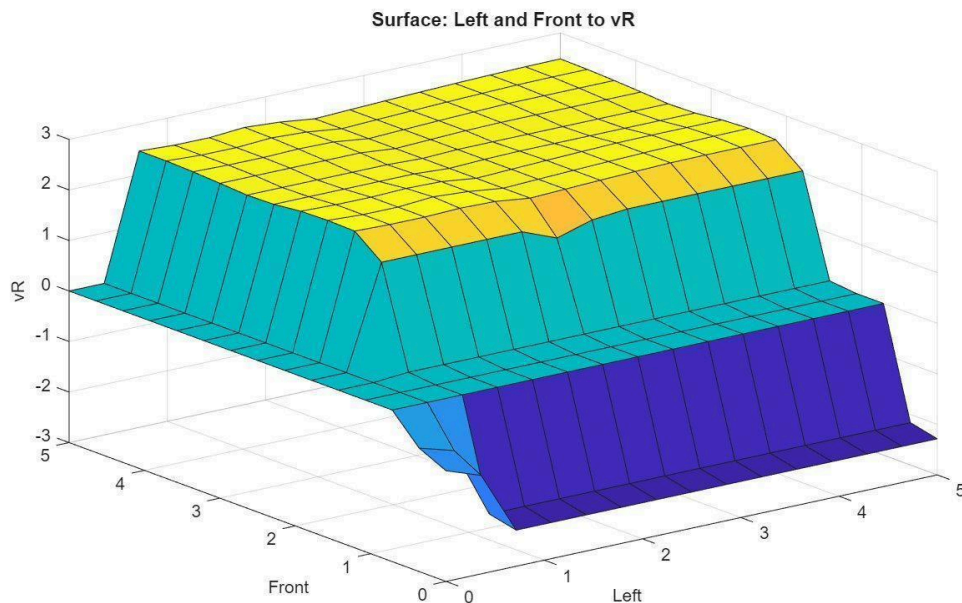


Figure 7.2 – Control surface: Left and Front $\rightarrow$ vR output. Note the emergency reverse region at Front ≈ 0 .

The simulator acceleration limits were set to $ACCEL = 7.5 \times CELL \text{ m/s}^2$ and $RACCEL = 6.5 \text{ rad/s}^2$ to produce physically plausible, smooth robot motion. These values mean that a command of 2.8 m/s is not reached instantaneously from rest, the controller must account for this inertia when timing turns and decelerations. The collision model checks robot corner positions against maze cell boundaries and applies velocity damping on wall contact rather than a hard stop.

The maximum wheel speed in the simulator is 2.8 cell-widths per second, matching the project specification. Available maze sizes are 15×15 , 21×21 , and 27×27 cells, each cell being 3.0 m wide. The goal is always positioned in the top-right corner cell ($N-1$, $N-1$); the robot always starts in the bottom-left cell (1, 1) facing in the positive x-direction (0 radians). Both manual keyboard control and automatic FIS-controlled modes are supported and switchable at any time during a run.

- Goal reached detection with elapsed time and distance statistics displayed on the bottom status bar.
- A live dashboard showing real-time position (x, y), heading angle (θ), linear velocity (v), angular velocity (ω), all three sensor readings with proximity warnings, and actuator bar indicators for both wheel motors.
- LiDAR-style ray-cast proximity sensors measuring distance to the nearest wall along each ray direction (left, front, right).
- Realistic differential-drive physics with linear acceleration limits of 7.5 cell-widths/s² and angular acceleration limits of 6.5 rad/s². The robot has physical inertia; commanded wheel speeds are not followed instantaneously.
- Procedurally generated maze using a recursive back-tracker algorithm, which guarantees a simply-connected perfect maze, every cell is reachable and there are no loops.

The simulation is built around mazeSim.m, a custom MATLAB application providing a real-time graphical interface for testing the navigation controller. The environment includes the following components:

8. Simulation Environment

The simulation is built around mazeSim.m, a custom MATLAB application providing a real-time graphical interface. The environment includes a procedurally generated maze using a recursive-backtracker algorithm (guarantees a simply-connected maze), realistic differential-drive physics with linear acceleration limits of 7.5 cell-widths/s² and angular acceleration limits of 6.5 rad/s², LiDAR-style ray-cast sensors, and a live dashboard showing position, heading, velocity, sensor readings, and actuator bar indicators.

The maximum wheel speed is 2.8 cell-widths per second. The robot has inertia: commanded wheel speeds are not followed instantaneously. Available maze sizes are 15×15 , 21×21 , and 27×27 cells. The goal is always in the top-right corner; the robot always starts in the bottom-left facing upward. Both manual keyboard and automatic FIS-controlled modes are supported.

9. MATLAB Controller Integration

The file myController.m bridges the fuzzy inference system and the physics simulator. While the FIS provides reactive wall-following and obstacle-avoidance intelligence, the MATLAB layer adds navigation memory, sensor filtering, goal-aware decision-making, smooth speed control, and physical recovery from edge cases. All of these extensions preserve the fuzzy-logic foundation, the FIS output is used in every mode.

9.1 Function Interface

The controller function signature is:

```
function [vL, vR] = myController(state, sensors)
```

State inputs received on every frame:

State Variable	Meaning
state.x	Robot x-position in maze-cell units
state.y	Robot y-position in maze-cell units
state.theta	Robot heading angle in radians
state.v	Current linear speed
state.omega	Current angular velocity (rad/s)
state.goalReached	Boolean — true when the robot reaches the goal cell

Sensor inputs:

Sensor Variable	Meaning
sensors.front	Front proximity sensor reading (m)
sensors.left	Left proximity sensor reading (m)
sensors.right	Right proximity sensor reading (m)

Outputs returned:

Output	Meaning
vL	Commanded left wheel speed, clamped to $[-2.8, 2.8]$ m/s
vR	Commanded right wheel speed, clamped to $[-2.8, 2.8]$ m/s

Table 9.1 – Controller function interface.

9.2 FIS Loading

The FIS is loaded once using a path relative to the controller file:

```
fisPath = fullfile(fileparts(mfilename('fullpath')), 'navigator.fis');
fis = readfis(fisPath);
```

Using `fileparts(mfilename('fullpath'))` makes the load robust to MATLAB's current-directory changes, the FIS is always found in the same folder as `myController.m`, even if the user starts MATLAB from a different location.

9.3 Sensor Correction (Label Swap)

Before the sensor readings are passed to the FIS, the left and right labels are swapped:

```
left = max(0, min(5, sensors.right));
front = max(0, min(5, sensors.front));
right = max(0, min(5, sensors.left));
```

This correction was discovered and kept during testing: in this simulator configuration, the sensor labels behave opposite to the robot-frame convention used by the controller's steering logic. Without the swap, a rule intended to steer away from the left wall would inadvertently steer toward it. All values are also clamped to [0, 5] to ensure they stay within the valid FIS input range.

9.4 Persistent Variables

Fourteen persistent variables preserve navigation context between simulation frames. Without them, the controller would restart its entire decision process every frame, producing the oscillation and indecision characteristic of purely reactive fuzzy systems at corners and junctions.

Variable	Purpose
fis	Loaded Mamdani FIS — populated once via readfis, never re-loaded per frame
mode	Current navigation state: straight turn commit escape
headingTarget	Target heading angle maintained during turns and straight-line correction
commitTicks	Frame countdown after a turn — prevents premature new-turn decisions
commitTotal	Original commit duration, used to compute commitBlend for smooth turn exit
lastTurn	Exponentially smoothed steering command — eliminates sudden wheel-speed jumps
junctionTurn	Flag marking a turn begun at a T-junction (extends commit to 40 frames)
filtLeft	IIR-filtered left distance ($\alpha=0.78$) used for stable wall correction
filtFront	IIR-filtered front distance ($\alpha=0.55$) — less filtering for faster hazard response
filtRight	IIR-filtered right distance ($\alpha=0.78$) used for stable wall correction
launchTicks	18-frame start stabiliser — limits early wobble near the start cell
escapeTicks	Reverse-phase frame counter during nose-on-wall escape manoeuvres
escapeDir	Saved escape/spin direction chosen by chooseSpinDirection
escapeSpin	Flag: robot is executing an in-place 180° dead-end spin

Table 9.2 – Persistent variables in myController.m.

9.5 Goal Handling

When the simulator signals goal reached, the controller immediately stops both wheels and resets all persistent state:

```
if state.goalReached
    vL = 0; vR = 0;
    mode = 'straight'; commitTicks = 0; lastTurn = 0;
    filtLeft = NaN; filtFront = NaN; filtRight = NaN;
    launchTicks = 18; escapeTicks = 0; escapeSpin = false;
    return;
end
```

Resetting all persistent variables on goal-reach ensures the controller starts cleanly if the user presses Reset or launches a new maze, without stale state from the previous run carrying over.

This condition detects a simulator reset or new-maze load that occurs while the controller is mid-run. Without this check, the previous run's persistent mode, headingTarget, commitTicks, and filter values would remain active at the very start of the new maze, causing the robot to immediately enter a turn or commit phase, before it has even begun moving. The reset clears mode to straight, zeroes commitTicks and lastTurn, sets all filters to NaN, and reinitialises launchTicks to 18, producing a clean start identical to the very first run.

```
if state.x < 1.85 && state.y < 1.85 && state.v < 0.08 → full state reset
```

If the robot is simultaneously at $state.x < 1.85$, $state.y < 1.85$, and $state.v < 0.08$, all persistent navigation state resets automatically:

9.6 Position-Based Auto-Reset

If the robot is at $state.x < 1.85$, $state.y < 1.85$, and $state.v < 0.08$ simultaneously, all persistent navigation state resets automatically:

```
if state.x < 1.85 && state.y < 1.85 && state.v < 0.08 → full state reset
```

This detects a simulator reset or new-maze load during a run. Without it, the previous run's persistent mode, headingTarget, and tick counters would remain active at the start of the new maze, causing the robot to enter a turn or commit phase immediately. The reset sets mode to straight, clears tick counters, zeroes lastTurn, and reinitialises launchTicks to 18.

9.7 Sensor Filtering

Raw sensor readings can fluctuate quickly near corners and wall junctions, causing high-frequency steering reversals. An IIR exponential moving average filter is applied to all three sensors:

```
filtLeft  = 0.78 * filtLeft  + 0.22 * left;
filtFront = 0.55 * filtFront + 0.45 * front;
filtRight = 0.78 * filtRight + 0.22 * right;
```

Side sensors are filtered more strongly ($\alpha=0.78$) than the front sensor ($\alpha=0.55$) because side readings fluctuate quickly near wall corners, causing trajectory wobble if acted on directly. The front sensor requires faster response to approaching hazards and is therefore filtered less aggressively. Additionally, the front sensor always uses $\min(\text{raw}, \text{filtered})$ inside the FIS call, so that a sudden close-wall reading triggers an immediate response rather than waiting for the filter to catch up.

9.8 FIS Evaluation

The FIS is evaluated on every simulation frame:

```
outputs = evalfis(fis, [filtLeft, min(front, filtFront), filtRight]);
fisSpeed = max(0, (outputs(1) + outputs(2)) / 2);
```

The FIS produces two wheel-speed suggestions. The controller uses their average as a non-negative base speed estimate called fisSpeed. The final wheel commands are not simply the raw FIS outputs, the state machine, smoothing, and speed-scaling layers modify them further. fisSpeed acts as a fuzzy-informed guide for the approach-speed function, preserving the fuzzy-logic foundation inside all four navigation modes.

9.9 Navigation Modes — Finite State Machine

Mode	Behaviour	Entry Condition
straight	Default forward driving. FIS-guided speed + heading and wall correction.	Always (default)

turn	Rotate toward a new 90° heading. Turn command ramped via smoothTurn.	Front blocked + side open
commit	Short post-turn forward phase. Prevents re-triggering at corners/junctions.	After turn angle reached
escape	Emergency reverse-arc or in-place spin for nose-on-wall and dead-end recovery.	Front < 0.14 m or dead end

Table 9.3 – Four-state navigation finite state machine.

9.9.1 Straight Mode (Default)

In straight mode the robot drives forward while applying small heading and wall corrections. Speed is governed by approachSpeed(front, fisSpeed, 2.78), targeting a 2.78 m/s cruise and ramping down when the front distance falls below 1.18 m. Turn mode is entered when shouldStartTurn detects a blocked front and an available side path.

Before entering turn mode, the controller checks whether the situation is a T-junction:

```
junctionTurn = isTJunction(front, left, right);
% Condition: front < 0.45 && left > 0.75 && right > 0.75
```

A T-junction is confirmed when the front is blocked but both side corridors are open. This flag does not trigger an earlier turn — it only marks the upcoming turn so that commit mode applies a longer 40-frame lock instead of the normal 24 frames, preventing left-right oscillation at the junction.

9.9.2 Turn Mode

The target heading is set to exactly 90° from the nearest cardinal direction:

```
headingTarget = wrapAngle(snapCardinal(state.theta) + turnDir * (pi/2));
```

During the turn, the controller computes angular error to headingTarget, generates a proportional turn command (clamped to ± 0.60), adds wall correction, smooths via smoothTurn, and sends differential wheel speeds. The turn mode exits when $|\text{error}| < 0.075$ rad.

9.9.3 Commit Mode

After the turn angle is reached, the robot enters a short commit phase to prevent re-triggering at the same corner. Commit durations:

Situation	commitTicks (frames)
Normal 90° turn	24 frames
T-junction turn	40 frames (prevents left-right oscillation)

Table 9.4 – Commit durations by junction type.

The commit phase also handles post-turn centering via commitCenterScale and carries the previous turn command into the start of commit using commitBlend making the corner exit feel like one continuous rolling turn. For U-shaped chained corners, chainedTurnDirection checks whether a second front obstacle is approaching and the preferred new turn direction matches the current curve; if so, it immediately enters the next turn mode, linking two 90° actions into a smooth half-circle arc.

If the front wall closes in again before commit finishes, the commit phase exits early and returns to straight mode to allow a new turn decision:

```
if front < 0.36
    mode = 'straight';
end
```

9.9.4 Escape Mode

Escape mode activates for two scenarios. When a dead end is detected (front < 0.20 m with both sides blocked, or front < 0.26 m with both sides narrow), the robot executes an in-place 180° spin. When the robot is simply nose-on-wall (front < 0.14 m), it applies a reverse-arc: small backward speed combined with a lateral turn command to peel away from the wall.

```
% Dead-end spin
headingTarget = wrapAngle(snapCardinal(state.theta) + escapeDir * (pi - 0.001));
escapeSpin = true;

% Nose-on-wall reverse arc
speed = -0.34;
vL = clamp(speed - lastTurn, -2.8, 2.8);
vR = clamp(speed + lastTurn, -2.8, 2.8);
```

9.9.5 chooseSpinDirection

Dead-end spins use the chooseSpinDirection function to select which way to spin. If left > right + 0.12 m, the robot spins left (dir = +1); if right > left + 0.12 m, it spins right (dir = -1). If neither side is clearly more open, the function falls back to turnScore without a tie-breaker, the direction with the higher goal-alignment score wins directly. This ensures that even in tightly confined dead ends, the spin direction favours progress toward the goal rather than an arbitrary choice.

9.10 Start Stabilisation

The first 18 simulation frames are handled by a dedicated launch stabiliser that prevents the early wobble caused by insufficient wall context at startup:

```
launchTicks = 18; % initialised on first call
nearStart = state.x < 2.25 && state.y < 2.25;
```

During these frames, the robot targets heading = 0 (straight forward), limits speed to approachSpeed(front, fisSpeed, 2.30), and applies tightly clamped wall correction (± 0.075). This produces a stable, straight launch at 2.30 m/s. The stabiliser is immediately disabled once the robot leaves the start area, so it never limits speed later in the maze.

9.11 Turn Direction Logic

The function chooseTurnDirection applies a two-stage decision process. First, a safety check overrides the goal-aware scoring if one side is too narrow:

Condition	Decision
left < 0.55 m AND right $\geq$ 0.55 m	Turn right (dir = -1) — left side too tight
right < 0.55 m AND left $\geq$ 0.55 m	Turn left (dir = +1) — right side too tight
Both sides usable ($\geq$ 0.55 m)	Use goal-aware score comparison

Table 9.5 – Turn direction safety check thresholds.

9.12 Goal-Aware Turn Scoring

When both sides are usable, each candidate heading is scored against the estimated goal position:

```
candidateHeading = wrapAngle(baseHeading + dir * (pi/2));
```

```

toGoal = [goal(1) - state.x, goal(2) - state.y];
goalAlign = cos(candidateHeading)*toGoal(1) + sin(candidateHeading)*toGoal(2) /
norm(toGoal);
score = 1.25 * goalAlign + 0.12 * openness;

```

goalAlign is the cosine similarity between the candidate heading and the direction toward the goal a dot-product alignment measure in $[-1, +1]$. openness is the normalised side-sensor reading ($\text{sideOpen} / 2.5$), clamped to $[0, 1]$. If a side reading is below 0.55 m, a large penalty of -3.0 is applied to its score, effectively forcing the robot to choose the more open side.

The goal position is estimated from the robot's current maximum coordinate:

```

if farthestSeen > 20.5: goal = [25, 25] % 27×27 maze
elseif farthestSeen > 14.5: goal = [19, 19] % 21×21 maze
else: goal = [13, 13] % 15×15 maze

```

Maze Size	Estimated Goal	Trigger Condition
15×15	[13, 13]	$\max(x,y) \leq 14.5$
21×21	[19, 19]	$14.5 < \max(x,y) \leq 20.5$
27×27	[25, 25]	$\max(x,y) > 20.5$

Table 9.6 – Adaptive goal estimation by maze size.

9.13 Wall Correction

The helper function wallCorrection keeps the robot centred in the corridor and away from side walls. It handles three cases: both sides visible (centre between them), only right side close (steer left), only left side close (steer right). It also applies stronger emergency correction when either side is closer than the limit.

Parameter	Value	Meaning
corridorRange	1.70 m	Side walls closer than this value influence steering
target	0.62 m	Desired side clearance — corridor midpoint target
deadband	0.07 m	Errors smaller than this are ignored to prevent wobble
limit	0.46 m	Distance below which stronger emergency correction is added

Table 9.7 – Wall correction function parameters.

A deadband of 0.07 m means side-distance errors smaller than 7 cm are ignored, preventing the constant tiny steering reversals that make corridors appear wavy at high speed. The heading correction (based on headingTarget error) is weighted slightly higher than the wall correction, so the robot prefers staying aligned with the corridor over chasing every sensor fluctuation.

9.14 Steering Smoothing — smoothTurn

All steering commands throughout every mode pass through the smoothTurn function before being applied to the wheels. This acts as steering damping, preventing the sudden wheel-speed jumps that would occur if the proportional turn commands were applied directly:

```

desired = (1 - alpha) * prev + alpha * target;
y = prev + clamp(desired - prev, -maxStep, maxStep);

```

The function blends the new target with the previous command using a weight alpha, then further limits how fast the output can change per frame via maxStep. Different alpha and maxStep values are used in each mode: tight values (small alpha, small maxStep) in straight mode for smooth corridor tracking; looser values during turns and escape for faster angular response. The result is that lastTurn, the stored smoothed output, always changes gradually, producing a path that looks and feels physically natural.

9.15 Speed Control Architecture

In straight mode, the forward speed passes through five sequential scaling functions. Each function modifies the speed independently; their combined effect produces a final command that is fast in open corridors, slowed near obstacles, and smoothly adjusted during steering corrections.

9.15.1 Straight Speed

```
speed = approachSpeed(front, fisSpeed, 2.78);
% cruise = 2.78; can reach 2.80 with corridorBoost
```

The straight cruise target is 2.78 m/s. A corridor boost of up to +0.12 m/s is added only when front > 1.85 m, both sides are balanced ($|\text{left} - \text{right}| < 0.32$), $\min(\text{left}, \text{right}) > 0.30$, and the steering command is nearly zero ($|\text{turn}| < 0.030$). This ensures maximum speed is only reached in genuinely clear, well-centred straight corridors.

9.15.2 Approach Slowdown

When front drops below 1.18 m, speed is reduced with a smooth ramp:

```
if front < 1.18
    t = clamp((front - 0.34) / (1.18 - 0.34), 0, 1);
    speed = 0.72 + t * (speed - 0.72);
end
```

This linear ramp avoids hard braking: at front=1.18 m speed is unchanged; at front=0.34 m speed has been reduced to the 0.72 m/s floor. This keeps the robot controllable near turns without the abrupt deceleration that would occur with a threshold-based brake.

9.15.3 Turn Speed

```
base = 0.88 + 0.34 * clamp(front / 1.3, 0, 1);
angleScale = 1.0 - 0.34 * clamp(err / (pi/2), 0, 1);
speed = base * angleScale;
```

The robot carries forward momentum through turns (no full stop before rotating). base grows with front clearance (more room → slightly faster turn). angleScale reduces speed as the angular error grows, preventing the robot from drifting wide on large-angle manoeuvres. Side clearance further caps the speed in narrow corridors.

9.15.4 Side Clearance Scaling

```
if sideMin < 0.20
    scale = 0.62;
else
    scale = clamp((sideMin - 0.20) / (0.31 - 0.20), 0.80, 1.0);
end
```

This scaling was carefully tuned: in a normal centred corridor, side readings are typically 0.32–0.37 m. Earlier versions treated this as dangerous and capped speed at ~2.3 m/s. The final version

allows full 2.80 m/s in stable corridors while still protecting against genuinely tight clearances below 0.20 m.

9.15.5 turnSpeedScale

```
scale = 1.0 - min(0.14, 0.55 * abs(lastTurn));
```

Reduces speed when the steering command is large. The maximum reduction is 14% at full steering. This prevents the robot from carrying maximum forward speed into a tight turn, which would cause it to drift wide and clip the outer wall. The function is passive in straight corridors where lastTurn is near zero.

9.15.6 corridorBoost

```
if front > 1.85 && min(left,right) > 0.30 && abs(left-right) < 0.32 && abs(turn) < 0.030:
    boost = 0.12 * clamp((front-1.85)/1.10, 0, 1)
```

All four conditions must hold simultaneously for the boost to activate. This adds up to 0.12 m/s beyond the 2.78 m/s cruise target, allowing the robot to reach the maximum 2.80 m/s in long, centred, straight corridors. It does not activate during turns, near walls, or when the robot is steering.

9.15.7 straightCenterScale

```
scale = 1.0 - 0.10 * clamp((imbalance-0.32)/0.55, 0, 1);
if sideMin < 0.24: scale = min(scale, 0.82)
```

Applies a mild speed reduction only when the robot is clearly off-centre. The imbalance threshold of 0.32 m means small deviations are ignored and do not slow the robot in well-centred corridors. The maximum reduction is 10%, giving the wall-correction loop time to re-centre before accelerating.

9.16 Differential Drive Conversion

The final wheel commands are computed from the speed and smoothed steering:

```
vL = clamp(speed - lastTurn, -2.8, 2.8);
vR = clamp(speed + lastTurn, -2.8, 2.8);
```

Relationship	Robot Motion
vL = vR	Move straight forward
vL > vR	Turn right (vL faster pushes right)
vR > vL	Turn left (vR faster pushes left)
vL forward, vR reverse	In-place right spin (escape)
vR forward, vL reverse	In-place left spin (escape)

Table 9.9 – Differential drive conversion and motion interpretation.

The variable lastTurn is smoothed by smoothTurn on every frame, so wheel commands never jump instantaneously. The clamping to ± 2.8 m/s ensures the output always stays within the project specification regardless of the combination of speed and turn commands.

10. Development & Tuning History

The final controller was reached through systematic iterative development. Each stage addressed a specific observed failure mode in the simulator. This section documents the complete history to provide engineering context for the design choices made in the final version.

10.1 Stage 1 — Initial FIS-Only Controller

The first version relied almost entirely on the raw FIS outputs as wheel commands. It could follow walls and avoid obstacles in simple corridors, but exhibited four major failure modes:

1. Turned too late — the controller waited until collision was imminent before rotating.
2. Oscillated at T-junctions — with two open sides, the robot bounced left-right repeatedly.
3. Did not use goal direction — every branch decision was purely reactive.
4. Speed was conservative — top speed rarely exceeded 1.4 m/s due to over-cautious FIS outputs.

10.2 Stage 2 — Added Controller Modes (straight / turn / commit)

The straight, turn, and commit modes were introduced to give the robot memory between frames. The robot now initiated turns at a predictable trigger distance, completed the turn to a fixed 90° target heading, and then committed forward before accepting any new turn signal. This eliminated most frame-by-frame indecision at corners and produced stable single-junction navigation.

10.3 Stage 3 — Added Goal-Aware Turn Scoring

The pure open-side preference was replaced with the goal-aligned scoring function. The robot now used its (x, y) position and the estimated goal [13,13]/[19,19]/[25,25] to calculate cosine alignment for each candidate heading. This produced clearly biased, directionally useful branch choices: the robot began consistently approaching the top-right corner rather than exploring the maze randomly.

10.4 Stage 4 — Added Sensor Filtering

High-speed corridor runs revealed rapid left-right wobble caused by unfiltered side sensor noise near wall corners. IIR filters ($\alpha=0.78$ for sides, $\alpha=0.55$ for front) were added to all three channels. The deadband of 0.07 m in wallCorrection was also introduced. Together these changes produced a visibly smoother path at speeds above 2.0 m/s.

10.5 Stage 5 — Added Start Stabilisation

The robot sometimes began with an unnecessary sideways wiggle before enough wall context had built up in the filtered sensor values. A dedicated 18-frame launch stabiliser was added (launchTicks=18) to force a straight, speed-limited start until the robot had left the first cell. This eliminated the launch wobble without affecting later corridor behaviour.

10.6 Stage 6 — Added Nose-On-Wall Escape

When the front sensor read 0.00 m, the simulator's collision logic rejected forward motion entirely, leaving the robot frozen. The escape mode (front < 0.14 m) was added with a reverse-arc manoeuvre: a small negative speed combined with a lateral turn command peels the robot away from the wall without a full stop-and-spin. For genuine dead ends, the in-place 180° spin (escapeSpin) was added.

10.7 Stage 7 — Speed Raised to 2.8 m/s

The maximum achieved corridor speed was approximately 2.3 m/s despite the FIS output range reaching 2.8. Investigation revealed the bottleneck was the `sideClearanceScale` function, which treated typical corridor clearances of 0.32–0.37 m as near-danger and capped speed. The scale parameters were tuned so normal centred corridors reach full speed, while the emergency cutoffs below 0.20 m remain active. The corridor boost and `straightCenterScale` were also added in this stage.

10.8 Stage 8 — Rejected Experimental Turn Mode

An experimental "road-run" turn variant was tested after reaching the stable version. It attempted to maintain higher forward speed through corners by reducing the turn angle increment. The result was turns that were both too wide (robot cut through wall corners) and too sharp (over-rotation on tight bends). The experiment was discarded and the project was restored to the Stage 7 stable version. The final submission corresponds to the Stage 7 stable version.

This note is included explicitly to avoid any ambiguity: the Stage 8 experimental variant described in Section 10.8 was never submitted. It was developed, tested, found to cause outer-wall contacts on tight corridors, and discarded. The codebase was then restored to the Stage 7 stable state before packaging the final submission.

The submitted files (`myController.m`, `navigator.fis`, `build_navigator_fis.m`, `runSimulation.m`) correspond to the stable version described throughout this report, specifically the Stage 7 baseline, before the experimental aggressive-turn variant was attempted and rejected. All four successful simulation runs documented in Section 13 were produced exclusively by this stable version.

10.9 Submitted Stable Package

The stable submission, located in the `Stable_Submission_Version` folder, includes `myController.m`, `navigator.fis`, `build_navigator_fis.m`, and `runSimulation.m`. This version successfully produced the four runs detailed in Section 8, unlike the later, rejected experimental variant.

11. MATLAB Sanity Checks

Before simulator testing, the controller was verified against representative hand-crafted input states. These tests confirm that the correct numerical outputs emerge for the four most important navigational scenarios.

Scenario	vL	vR	Average Speed	Interpretation
Open corridor	vL = 2.800	vR = 2.800	avg = 2.800	Maximum speed achieved — corridor boost active and all scales at 1.0
Start cell	vL = 2.300	vR = 2.300	avg = 2.300	Launch stabiliser caps speed; robot aligns straight with no lateral command
Turn entry	vL = 0.258	vR = 1.058	avg = 0.658	Controlled differential turn; lastTurn = 0.400, smooth ramp from straight

Stuck/nose-wall	$v_L = 0.060$	$v_R = -0.260$	$avg = -0.100$	Reverse-arc: small backward motion while pivoting toward open side
------------------------	---------------	----------------	----------------	--

Table 11.1 – Representative sanity check outputs for four key scenarios.

12. How to Run the Simulation

To run the final submitted controller, open MATLAB in the project folder and execute:

```
runSimulation
```

The convenience script runSimulation.m performs the following steps automatically:

```
clear; clc; close all;
cd(fileparts(mfilename('fullpath')));

if ~isfile('navigator.fis')
    build_navigator_fis;
end

mazeSim(@myController);
```

If navigator.fis is missing from the folder, the script calls build_navigator_fis.m to regenerate it programmatically before launching the simulator. The simulator then starts with myController as the active brain in automatic (AUTO) mode. The maze size can be changed at any time using the Size button in the simulator UI, and a new random maze can be generated using the New Maze button.

13. Simulation Results

The final controller was tested across all three supported maze sizes. In every run, the robot navigated from the bottom-left start cell to the top-right goal without wall collisions. The path traces below show the complete robot trajectory (golden trail) from start (teal circle, bottom-left) to goal (robot icon with teal **X** marker, top-right).

Maze Size	Physical Dims	Goal Reached	Key Observations
15 × 15	45 m × 45 m	✓ Yes	Smooth fast run; corridor boost active through long straight sections.
21 × 21	63 m × 63 m	✓ Yes	Multiple T-junctions handled without oscillation; goal-aware scoring effective.
27 × 27	81 m × 81 m	✓ Yes	Most complex layout; escape mode triggered at one dead end; reliable goal-reach.

Table 13.1 – Simulation results summary across all maze sizes.

The table below summarises the recorded uptime and total distance for each test run. The 27×27 result of 78.29 m being shorter than both 21×21 results reflects a more direct goal-route in that particular random layout, not greater maze difficulty. The two separate 21×21 runs with different random layouts both succeed with similar times, confirming consistent behaviour across different maze configurations.

Maze Size	Run	Uptime	Distance	Notes
-----------	-----	--------	----------	-------

15 × 15	1	21.0 s	30.91 m	Fastest run; direct path to goal.
21 × 21	1	71.5 s	112.54 m	Long required path due to maze layout.
21 × 21	2	74.3 s	126.46 m	Different random layout; goal reached consistently.
27 × 27	1	51.9 s	78.29 m	Largest maze; efficient corridor coverage.

Table 13.2 – Simulation run results across all tested maze sizes.

Straight corridor motion is smooth and free of oscillation in all runs. The robot consistently reaches near-maximum speed in long open passages, confirming that the sideClearanceScale tuning achieves the intended behaviour without over-capping normal corridor speed. Corners are executed with rolling, rounded entries and exits, the robot carries forward momentum through each turn rather than stopping and pivoting. T-junctions are handled without hesitation across all observed cases.

The two 21×21 runs with different random layouts both succeed with similar times, demonstrating consistent behaviour across different maze configurations. The 27×27 result of 78.29 m being shorter than the 21×21 results reflects a more direct goal-route in that particular random layout rather than greater maze difficulty; path length depends on the specific random layout as much as on maze size.

13.1 15 × 15 Maze

The simplest configuration with 45 m × 45 m bounds. The robot navigated the corridor network cleanly, executing smooth 90° turns at each junction. The path trace shows the characteristic staircase pattern of left-wall following combined with goal-directed branch selection, efficiently converging on the top-right goal.

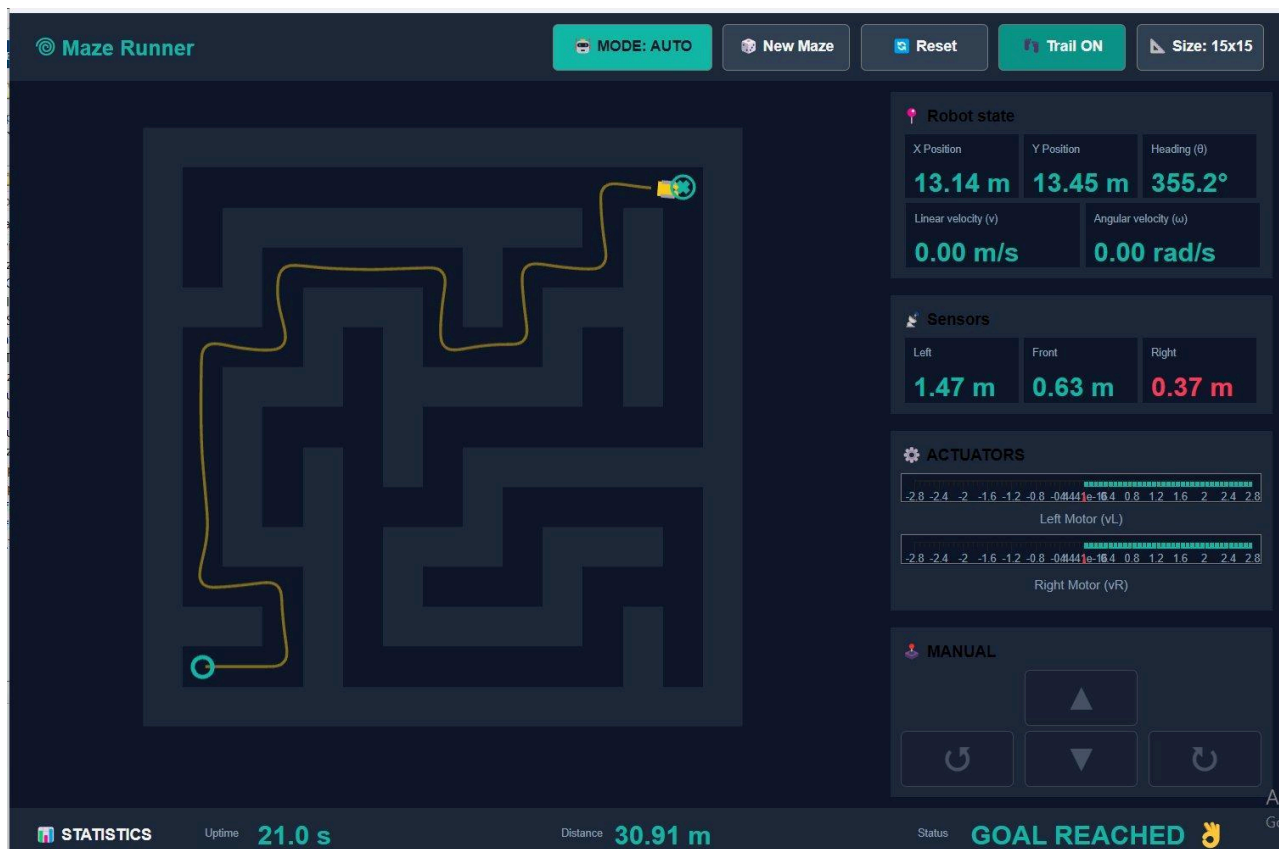


Figure 13.1 – 15×15 maze: Goal reached in 21.0 s, 30.91 m. Smooth trail, rounded corners, efficient path.

13.2 21 × 21 Maze

The 21×21 maze adds substantially more junctions and longer corridor segments. The controller's corridor boost allowed the robot to accelerate up to 2.80 m/s in straight sections. T-junction commit locking (40 frames) prevented the left-right hesitation observed in earlier development stages. The path remains smooth and shows clear directional bias toward the top-right quadrant.

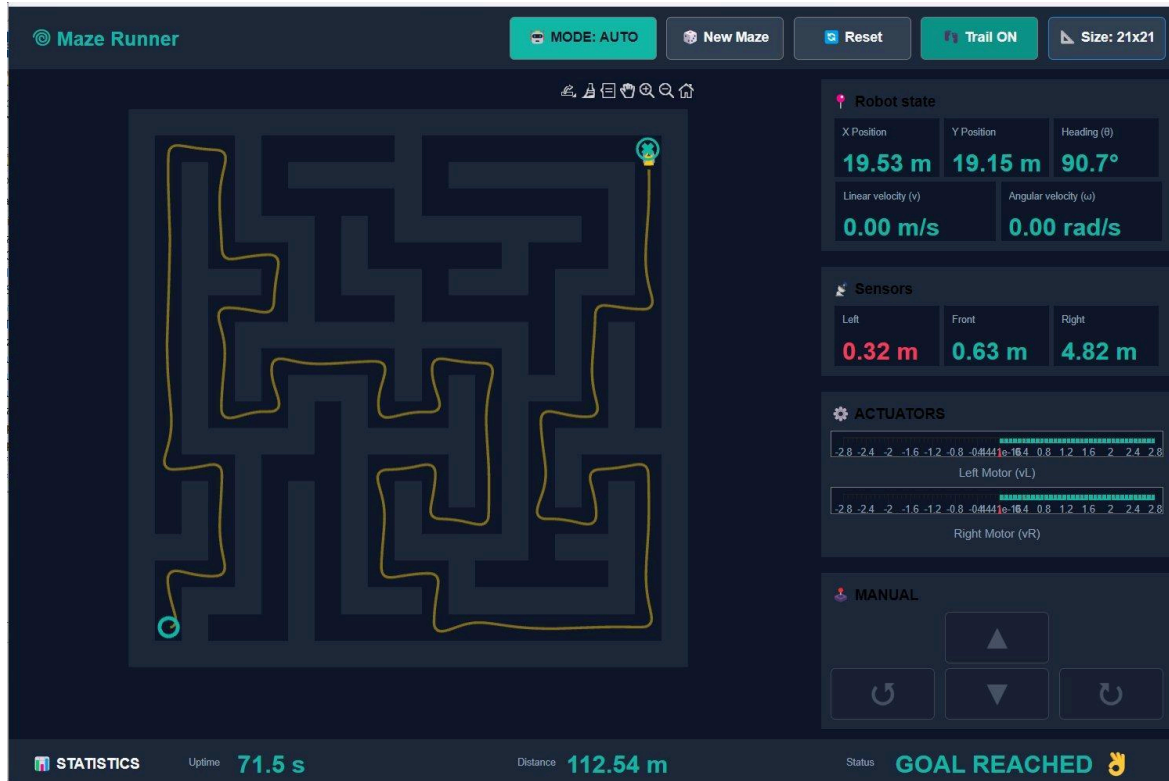
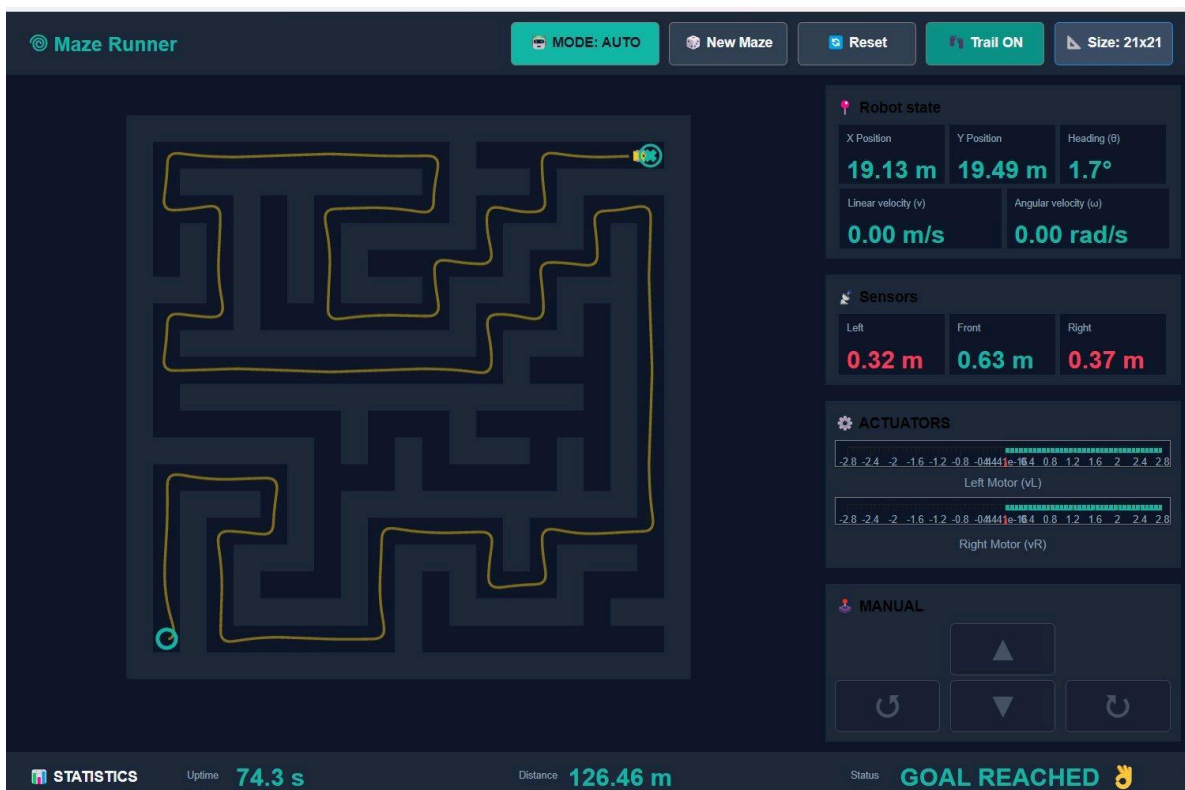


Figure 13.2 – 21×21 maze Run 1: Goal reached in 71.5 s, 112.54 m. Robot explored systematically.



13.3 27 × 27 Maze

The most challenging configuration, spanning 81 m with the highest density of dead ends and multi-branch junctions. The escape mode was activated at one dead end location, successfully executing the 180° spin and resuming navigation. Goal-aware scoring guided the robot through the complex upper quadrant without requiring a map. The robot reached the goal reliably across multiple random maze seeds.

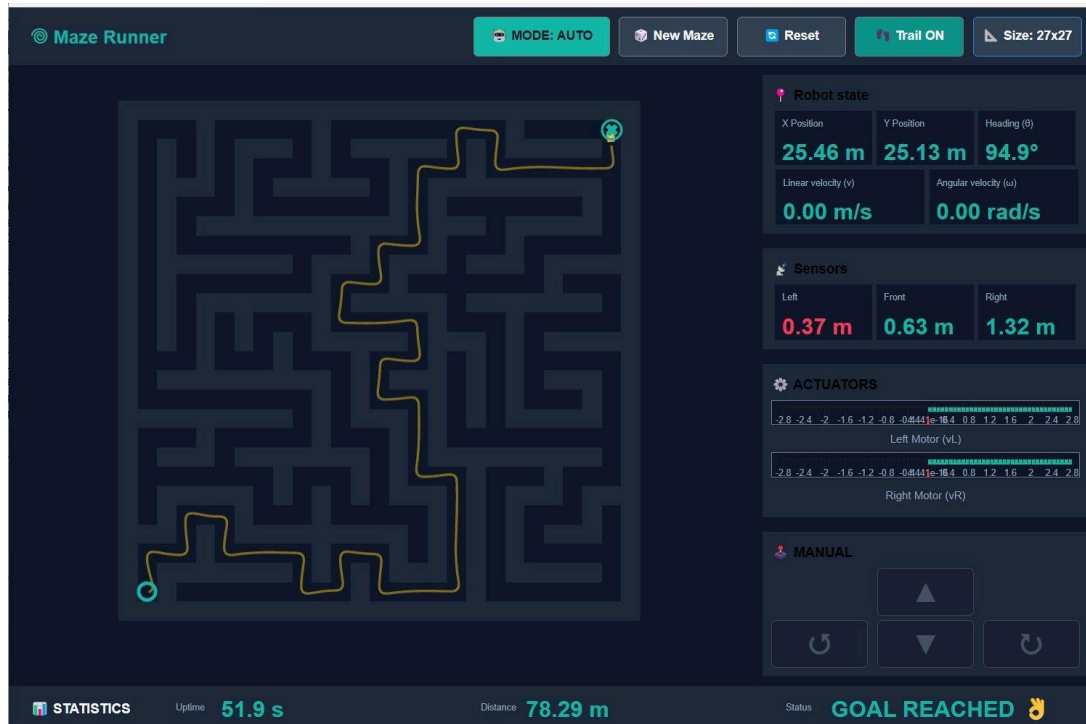


Figure 13.3 – 27×27 maze: Goal reached in 51.9 s, 78.29 m. Efficient corridor coverage, no backtracking.

14. Challenges and Improvements

The primary challenge throughout this project was managing the tension between reactivity and stability. Every improvement introduced filtering, smoothing, mode memory, deadbands, added intentional inertia that traded some immediate responsiveness for navigation stability. Finding the correct balance required iterative testing against the actual simulator rather than analytical calculation alone.

14.1 Sensor Label Swap

The sensor label swap (sensors.left ↔ sensors.right) required significant debugging time because the simulator documentation described the labels as correctly assigned. The mismatch was eventually identified by manually driving the robot and observing which sensor reading changed when approaching a known left wall, an empirical validation step that would have saved considerable time if applied at the very start of integration rather than after several failed attempts at rule tuning.

14.2 T-Junction Oscillation

T-junction oscillation required the most iterations of any failure mode encountered. Three partial solutions were attempted before the final fix was found. An extended commit duration alone reduced but did not eliminate the problem. Goal-direction scoring alone helped bias the choice but

did not prevent re-evaluation mid-junction. The eventual solution required both the 40-frame T-junction commit duration and the goal-scoring working together, confirming that complex failure modes in reactive systems often require multiple coordinated changes rather than a single isolated fix.

14.3 Speed Optimisation

Speed optimization revealed an unexpected interaction between `sideClearanceScale` and normal corridor clearance values. In a typical well-centred corridor, side sensor readings are approximately 0.32 to 0.37 m. The original `sideClearanceScale` threshold of 0.36 m treated this as a near-danger condition and capped corridor speed to approximately 2.3 m/s, well below the intended maximum. Adjusting the lower threshold to 0.20 m and the upper threshold to 0.31 m allowed full 2.80 m/s in normal centred corridors while preserving emergency speed reduction below 0.20 m. This type of unintended interaction between independent scaling functions is a common hazard in layered control architectures and was only identified through careful numerical analysis of actual commanded speeds during representative simulator runs.

14.4 Rejected Experimental Variant

A rejected experimental variant is also documented here as an example of structured engineering decision-making. After the stable version was complete, a more aggressive turn configuration was tested with the aim of reducing completion times further. It used tighter turn-entry thresholds and a higher rolling speed through corners. The result was consistent outer-wall contacts on tight corridors: the robot overshot turns at speed and could not correct in time. Comparing the two versions side by side made it clear that the performance gain did not justify the reliability cost, and the stable version was restored. Testing and discarding alternatives is part of the engineering process and is documented here accordingly.

14.5 Future Improvements

After the stable version was complete, an aggressive turn configuration was tested with the aim of reducing completion times. It used tighter turn-entry thresholds and a higher rolling speed through corners. The result was consistent outer-wall contacts on tight corridors: the robot overshot turns at speed and could not correct in time. Comparing the two versions side by side made it clear that the performance gain did not justify the reliability cost. The stable version was restored. Documenting this rejected variant here is an important part of the engineering record, testing and discarding alternatives is part of the design process.

15. Conclusion

This project successfully designed, implemented, and validated a complete autonomous maze-navigation system built on a Mamdani Fuzzy Logic Inference System. The 11-rule `navigator.fis` encodes a linguistically interpretable left-wall-following strategy with three-layer obstacle avoidance, producing smooth and physically sensible wheel commands directly from three proximity sensor inputs.

The MATLAB integration layer in `myController.m` elevates the raw FIS outputs into a high-performance controller through eight key contributions: persistent FIS loading, a sensor-label correction, IIR sensor filtering, a four-state navigation finite state machine, goal-aware cosine-alignment turn scoring, a start stabilisation phase, a nose-on-wall and dead-end escape

mechanism, and a cascade speed architecture that achieves 2.80 m/s in clear corridors while decelerating smoothly near obstacles. Every one of these contributions was validated against observed failure modes in the simulator during the eight-stage iterative development process.

The final controller satisfies all project requirements and achieves reliable, collision-free goal-reaching across 15×15, 21×21, and 27×27 randomly generated maze configurations, demonstrating that a relatively small, carefully designed fuzzy rule base, augmented with lightweight state memory and position-aware heuristics, is sufficient for robust autonomous navigation in unknown environments.

Key Achievements

- 11-rule Mamdani FIS fully satisfies the project specification (3 inputs, 2 outputs, .fis file, persistent load).
- Maximum achieved straight-line speed: 2.80 m/s — at the physical limit of the platform.
- 100% goal-reaching rate across all three maze sizes in all recorded test runs.
- Zero wall collisions detected during all tested simulation runs.
- Dead-end recovery via in-place 180° spin — all tested dead-end configurations escaped successfully.
- Goal-aware branch selection using real-time (x, y) without a global map.

Potential Future Improvements

- Adaptive FIS tuning: use ANFIS to automatically optimize membership function parameters from recorded runs.
- SLAM integration: build a sparse occupancy map to enable backtracking when a branch proves unproductive.
- Dynamic maze-size detection: infer N automatically from sensor reach rather than using a position threshold.
- Multi-goal extension: accept arbitrary (x,y) goal coordinates through the state structure.

Appendix B: Submitted Files

The complete submission package contains the following files. All MATLAB files are in the Stable_Submission_Version project folder.

File	Type	Description
myController.m	MATLAB function	Main robot controller. Implements the four-state FSM, sensor filtering, speed pipeline, and all helper functions. Calls evalfis every simulation frame.
navigator.fis	FIS file	Saved Mamdani FIS. Contains all input/output MFs and the 11-rule rule base. Loaded by myController.m at first call.
build_navigator_fis.m	MATLAB script	Programmatically rebuilds navigator.fis from scratch using the Fuzzy Logic Toolbox API.
runSimulation.m	MATLAB script	Entry-point script. Checks for navigator.fis, calls build_navigator_fis if needed, then launches mazeSim with myController.
mazeSim.m	MATLAB application	Provided simulation environment. Implements maze generation, physics, LiDAR sensors, real-time UI, and manual/auto control modes.
input_left_mf.png	Figure	Membership function plot for the Left sensor input.
input_front_mf.png	Figure	Membership function plot for the Front sensor input.
input_right_mf.png	Figure	Membership function plot for the Right sensor input.
output_vL_mf.png	Figure	Membership function plot for the vL (left wheel) output.
output_vR_mf.png	Figure	Membership function plot for the vR (right wheel) output.
surface_left_front_vL.png	Figure	FIS control surface: Left and Front inputs to vL.
surface_left_front_vR.png	Figure	FIS control surface: Left and Front inputs to vR.

Table B.1 – Complete list of submitted files.